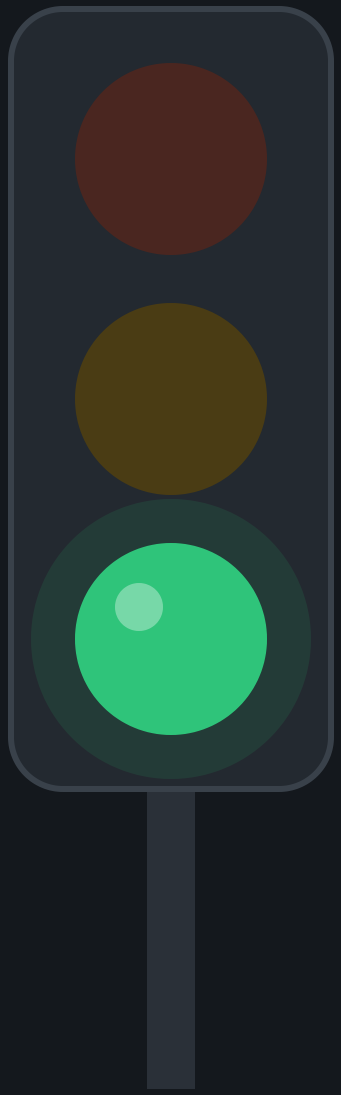
FOAI PBL PRESENTATION | REVIEW 1 AND 2

Intelligent Traffic Signal Control Using Fuzzy Logic and Optimization

A signal that looks at the queue before it decides how long to stay green.



PRESENTED BY

Mohammed Amaan Khan

25WU0101075 | B.Tech CSE, 2nd Year | Rhinos

GUIDED BY

Dr. Bhargav Prajwal Pathri

Foundations of AI | Individual project

00 ROADMAP

What I will cover in the next few minutes

The talk follows the Review 1 evaluation points in order.

01

Problem and Objectives

Why fixed timers fail, the exact problem I am solving and what I aim to deliver.

02

Literature Review

What earlier researchers built, what worked, and the gap that is still open.

03

Proposed Methodology

Simulator, fuzzy controller, Genetic Algorithm tuning and how I will measure it.

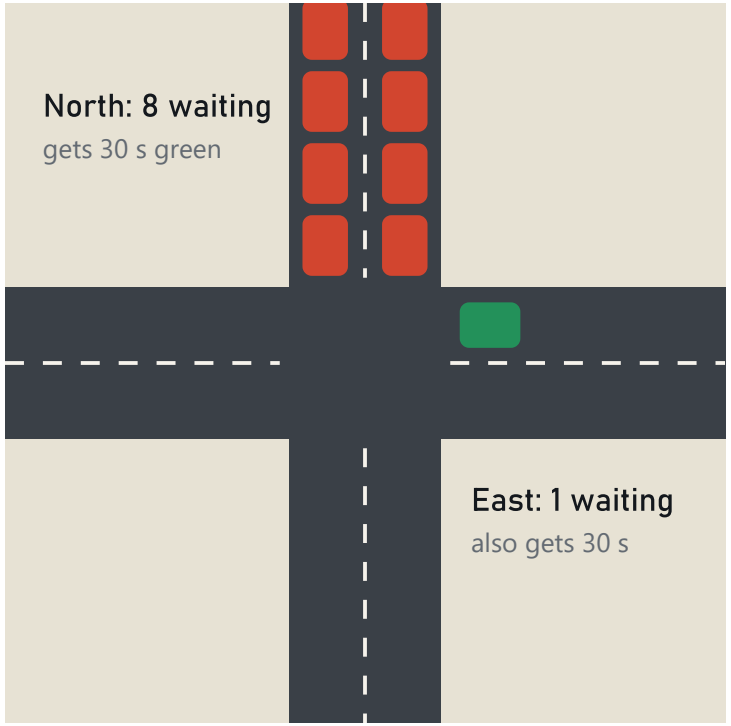
04

Plan for Review 2

Work phases, tools and the results I expect to show next time.

A fixed timer cannot see the road

Most signals give every direction the same green time, whether the road is full or empty.



Illustrative example of one signal cycle

Green time is wasted

The empty road holds a green light while the crowded road keeps waiting on red.

Traffic keeps changing

Morning peak, afternoon lull, a sudden event. One fixed plan cannot suit all of them.

The cost is real

Longer waiting means more fuel burnt while idling, more emissions and delayed emergency vehicles.

A traffic police officer does this job better because they **judge** the queue as short, medium or long. That human style of reasoning is exactly what fuzzy logic models.

Problem statement

To design a traffic signal controller for a four-way intersection that decides green time from the live queue on each road using fuzzy logic, and to tune that controller with an optimization algorithm so that the average waiting time of vehicles is as low as possible.

In scope

- One isolated four-way intersection
- Simulated traffic with random arrivals
- Fuzzy controller plus Genetic Algorithm tuning
- Comparison against a fixed-time signal

Out of scope

- Real cameras, sensors or road hardware
- A network of many connected junctions
- Pedestrian phases and turning lanes
- Deployment on a real controller

Objectives

Five things I will build and measure. Each one can be checked at Review 2.

1. **Simulate** a four-way signalised intersection in Python with random vehicle arrivals on each road.
2. **Design** a Mamdani fuzzy controller that reads queue lengths and outputs how long to extend the green.
3. **Optimize** the membership functions of that controller with a Genetic Algorithm to minimise average waiting time.
4. **Compare** three controllers on the same traffic: fixed-time, fuzzy, and GA-tuned fuzzy.
5. **Test** under light, heavy and unbalanced traffic to see where each controller wins or fails.

What has already been done

Work	Approach	Contribution	Limitation
Webster, 1958 [1]	Fixed-time formula	Gave a formula for the best cycle length and green split from average flow.	Assumes steady demand. Cannot react to sudden changes.
Pappis and Mamdani, 1977 [3]	Fuzzy controller	First fuzzy logic controller for a traffic junction. Reported lower delay than a vehicle-actuated controller.	Simple junction of two one-way roads. Rules written by hand.
Hunt et al., 1981 [4]	SCOOT adaptive system	Adjusts signal timings online from road detector data across a city network.	Needs detectors and central infrastructure. Costly to install.
Karr, 1991 [5]	GA + fuzzy	Showed that a Genetic Algorithm can tune fuzzy membership functions automatically.	Shown on general control problems, not on traffic.
Trabia et al., 1999 [6]	Two-stage fuzzy	Split the decision into two fuzzy stages for an isolated intersection with turning traffic.	Membership functions still set by expert judgement.
Niittymaki and Pursula, 2000 [7]	Fuzzy vs actuated	Simulation study showing fuzzy control can compete with traditional vehicle-actuated control.	Rule base tuned manually for the tested site.

What the literature tells me, and the gap

A

Fuzzy logic fits the problem

Queue length is naturally vague. "Long queue" has no sharp cut-off, and fuzzy sets handle that well.

B

Hand tuning is the weak point

In most fuzzy signal papers the shapes of the membership functions come from expert guesswork.

C

Optimizers can do the tuning

GA and PSO search for good parameters without needing a formula for the objective.

THE GAP I AM ADDRESSING

A clear side by side study, on one simple and reproducible simulator, of how much a fuzzy signal improves over a fixed timer, and how much more is gained when its membership functions are tuned by a Genetic Algorithm instead of by hand.

Course concepts used: Intelligent agents | Fuzzy sets and inference | Genetic Algorithm | Objective functions

System overview

The signal works as an agent: it senses the queues, reasons with fuzzy rules, then acts on the lights.

1. Traffic simulator
Four-way intersection. Vehicles arrive randomly on each road (Poisson arrivals).

→

2. Sensing
Queue on the road that has green. Longest queue among roads on red.

→

3. Fuzzy controller
Fuzzify inputs, apply rules, defuzzify with the centroid method.

→

4. Action
Extend the current green by the computed seconds, or switch phase.

↑ ↓

Optimization loop (offline)

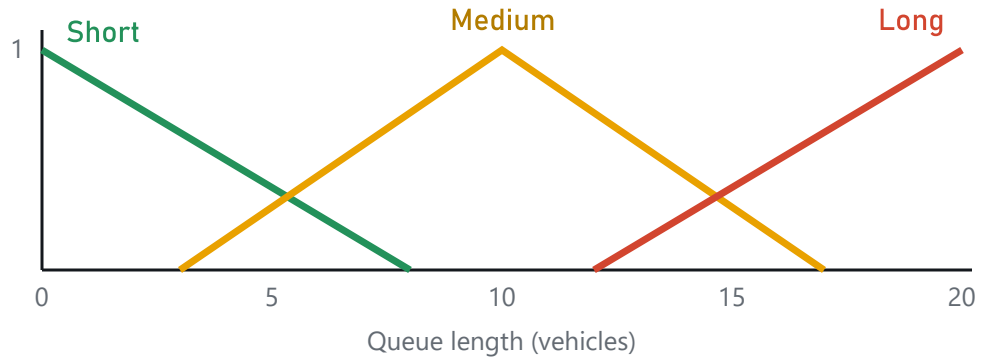
A **Genetic Algorithm** proposes new membership function shapes, runs the simulator with them, and keeps the ones that give the **lowest average waiting time**. The best set is then fixed inside the fuzzy controller.

Fuzzy controller design

Variables

Input 1: queue on the green road (vehicles)
Input 2: queue on the waiting roads (vehicles)
Output: green extension (seconds)
Each one has three fuzzy sets: Short, Medium, Long.

Membership functions (initial shapes)



Rule base (9 rules)

Cell value is the green extension.

Green road queue	Waiting: Short	Waiting: Medium	Waiting: Long
Short	Short	Short	Short
Medium	Medium	Medium	Short
Long	Long	Medium	Medium

Example: IF green road queue is **Long** AND waiting queue is **Short** THEN extension is **Long**.

Inference: Mamdani, AND as minimum.
Defuzzification: centroid, giving one crisp value in seconds.

Optimization layer: Genetic Algorithm

The rules stay the same. The GA only searches for better membership function shapes.

1. Encode
Chromosome = the corner points of all triangular membership functions.

↓

2. Evaluate
Run the simulator with each chromosome and record average waiting time.

↓

3. Select
Keep the chromosomes with the lowest waiting time (tournament selection).

↓

4. Crossover and mutate
Mix good parents and add small random changes to explore new shapes.

↻ repeat for a fixed number of generations

Fitness function

Minimise: average waiting time per vehicle

measured over one full simulation run, with the same traffic seed for every candidate so the comparison is fair.

Why a Genetic Algorithm?

The fitness comes out of a simulation, so there is no equation to differentiate. Gradient descent cannot be applied directly. GA only needs the score of each candidate.

Constraint handling

Corner points are kept in order (Short before Medium before Long) so every candidate is still a valid fuzzy set.

How I will evaluate it

Controller	How it decides green time	Role
Fixed-time	Same green for every road, every cycle	Baseline
Fuzzy (hand tuned)	Fuzzy rules with my initial membership functions	Proposed, stage 1
Fuzzy + GA	Same rules, membership functions tuned by GA	Proposed, stage 2

Traffic scenarios

Light: low arrival rate on all roads.
Heavy: high arrival rate on all roads.
Unbalanced: one busy road, three quiet roads. This is where a fixed timer should struggle most.

Metrics

- Average waiting time per vehicle (main metric)
- Average and maximum queue length
- Throughput: vehicles cleared in the run

Tools

Python | NumPy | scikit-fuzzy | Matplotlib | Custom GA code

Fair testing: all three controllers face the same random traffic, repeated over several seeds, and I report the average.

04 PLAN FOR REVIEW 2

Work plan and expected outcome

1

Build the simulator

Intersection model, arrivals, queues and the fixed-time baseline.

2

Add the fuzzy controller

Membership functions, nine rules, centroid defuzzification.

3

Tune with GA

Encode, run generations, plot the fitness curve.

4

Compare and report

Three controllers, three scenarios, graphs and conclusion.

What I expect to show

- Fuzzy control waits less than the fixed timer, most clearly in unbalanced traffic
- GA tuning gives a further drop over the hand-tuned version
- A fitness curve that improves across generations

Deliverables for Review 2

- Working Python code with a live demo run
- Comparison graphs and a results table
- Conclusion, limits found and future scope

REVIEW 2

From plan to working system

Everything proposed in Review 1 is now built, run and measured.

Implementation
What I built and how it runs

Results and Analysis
Measured on unseen traffic

Conclusion
What the project shows

Future Scope
Limits and next steps

What I built

One Python program of about 220 lines, using only NumPy. The full experiment runs in about one minute on my laptop.

1. Intersection simulator

Four roads, one-second time steps, Poisson arrivals. One vehicle leaves every 2 s of green. Every phase change costs 3 s of lost time.

2. Fuzzy engine

Mamdani inference with the nine rules from Review 1, AND as minimum, centroid defuzzification. Output is the green extension in seconds.

3. Genetic Algorithm

Chromosome of 15 numbers (5 per fuzzy variable). Tournament selection, uniform crossover, Gaussian mutation, elitism and a repair step.

4. Experiment runner

Trains on one set of traffic seeds, then tests all controllers on ten unseen seeds per scenario and saves every metric.

ONE CHANGE FROM THE PLAN

I planned to use scikit-fuzzy, but the GA needs about 11,000 simulation runs. So I wrote the fuzzy engine in NumPy and precompute each controller into a 21 x 21 lookup table, which makes every run fast.

Experimental setup

Simulator	
Run length	3600 s (one hour of traffic)
Discharge rate	1 vehicle per 2 s of green
Lost time per switch	3 s
Fuzzy green limits	Minimum 6 s, maximum 60 s
Switch rule	Change phase if extension is under 4 s

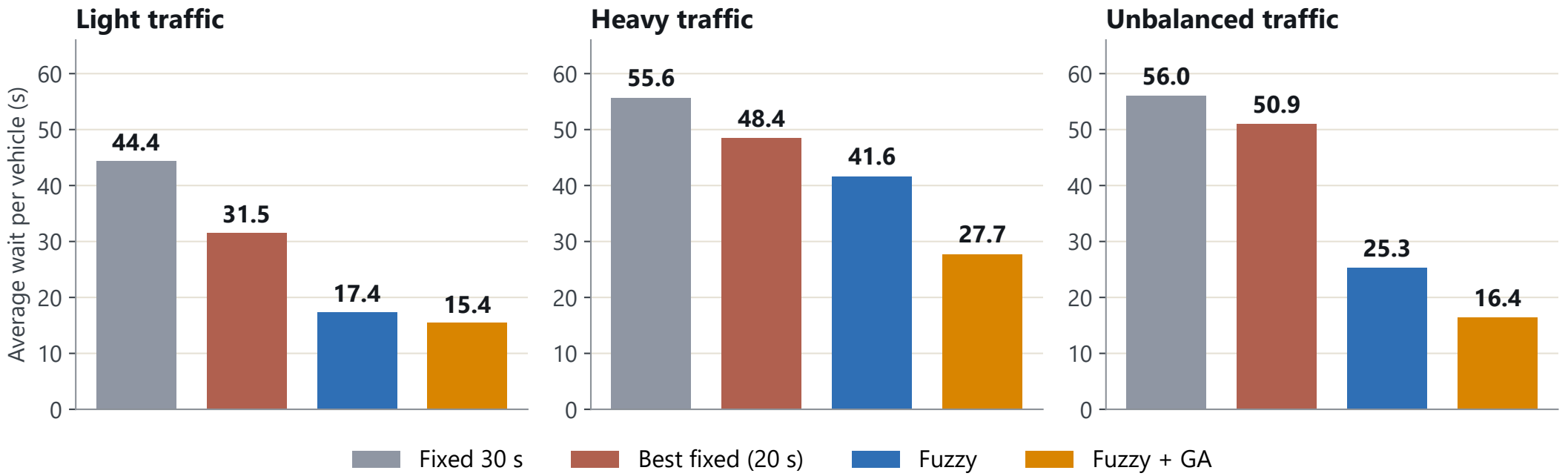
Scenario	Arrivals per road (vehicles per hour)
Light	144 on every road
Heavy	324 on every road
Unbalanced	360 on one road, 108 on the other three

Genetic Algorithm	
Population	30 candidates
Generations	40
Selection	Tournament of 3, best 2 kept (elitism)
Crossover	Uniform, gene by gene
Mutation	20% of genes, Gaussian, sigma 1.0
Fitness	Mean wait over 3 scenarios x 3 seeds

Train and test are separate

The GA only ever sees 3 training seeds per scenario. Every result in the next slides is the average over **10 different seeds it never saw**.

Result: waiting time drops in every scenario



65% lower wait than Fixed 30 s in **light** traffic

50% lower wait than Fixed 30 s in **heavy** traffic

71% lower wait than Fixed 30 s in **unbalanced** traffic

Full results table

Wait = average seconds per vehicle. Avg queue = vehicles waiting across all roads. Max queue = longest single-road queue.

Controller	Light			Heavy			Unbalanced		
	Wait (s)	Avg queue	Max queue	Wait (s)	Avg queue	Max queue	Wait (s)	Avg queue	Max queue
Fixed 30 s	44.4	7.2	10.5	55.6	20.2	20.8	56.0	10.8	21.0
Best fixed (20 s)	31.5	5.1	8.8	48.4	17.6	19.9	50.9	9.8	20.1
Fuzzy (hand-set)	17.4	2.8	6.5	41.6	15.2	14.0	25.3	4.9	10.2
Fuzzy + GA	**15.4**	**2.5**	**6.0**	**27.7**	**10.1**	**12.8**	**16.4**	**3.2**	**8.3**

Against the stronger baseline

"Best fixed" is the best green time found by sweeping 5 to 60 s. Fuzzy + GA still waits **51%, 43% and 68%** less than it in the three scenarios.

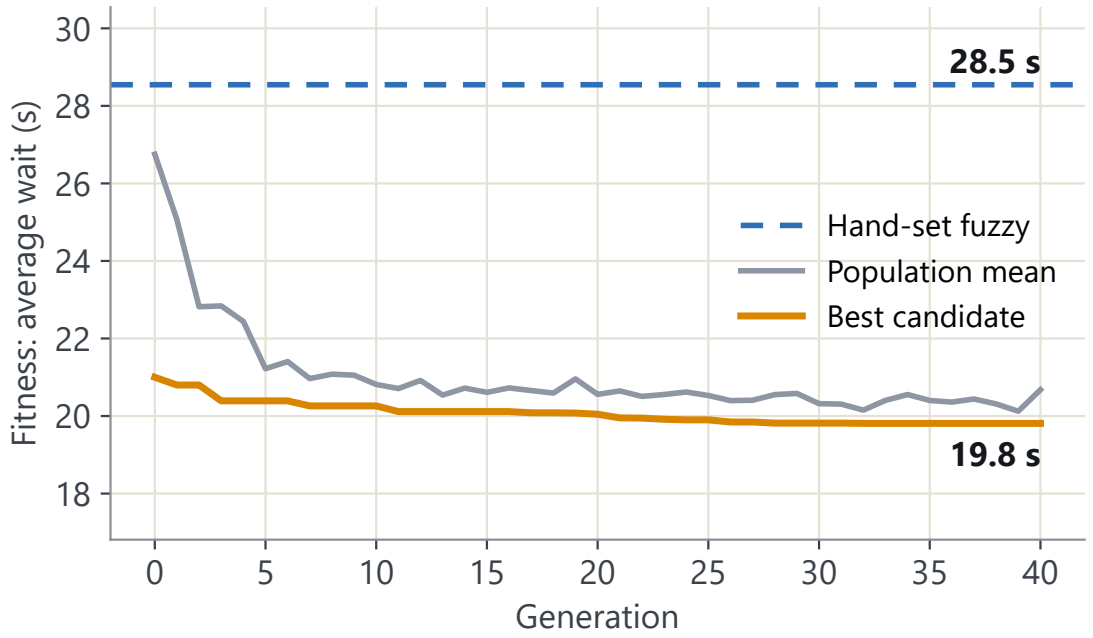
What the GA adds

Over the hand-set fuzzy controller, GA tuning cuts waiting by a further **11%** (light), **33%** (heavy) and **35%** (unbalanced).

Throughput

Almost equal for all four (within 1%). Demand is below capacity, so every controller clears the vehicles. The gain is in **delay**, not volume.

How the Genetic Algorithm learned



Fitness on training traffic, lower is better

Training fitness: 28.5 s to 19.8 s

That is 31% lower than my hand-set membership functions on the same training traffic.

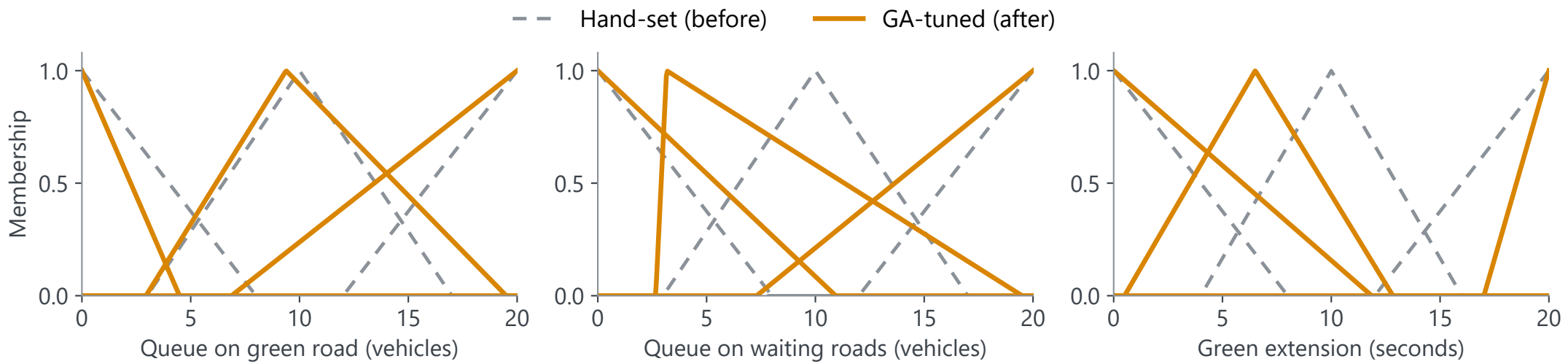
Most of the gain came early

The very first random population already held a candidate at 21.0 s. This shows how far from optimal my manual shapes were. The GA then refined it to 19.8 s.

No overfitting

Training score 19.8 s, unseen test score 19.8 s (mean of the three scenarios). The tuned controller generalises to new traffic.

What the GA actually changed



Green road: "Long" starts sooner

A queue now starts counting as Long from 7 vehicles instead of 12, so a busy road earns a long green earlier.

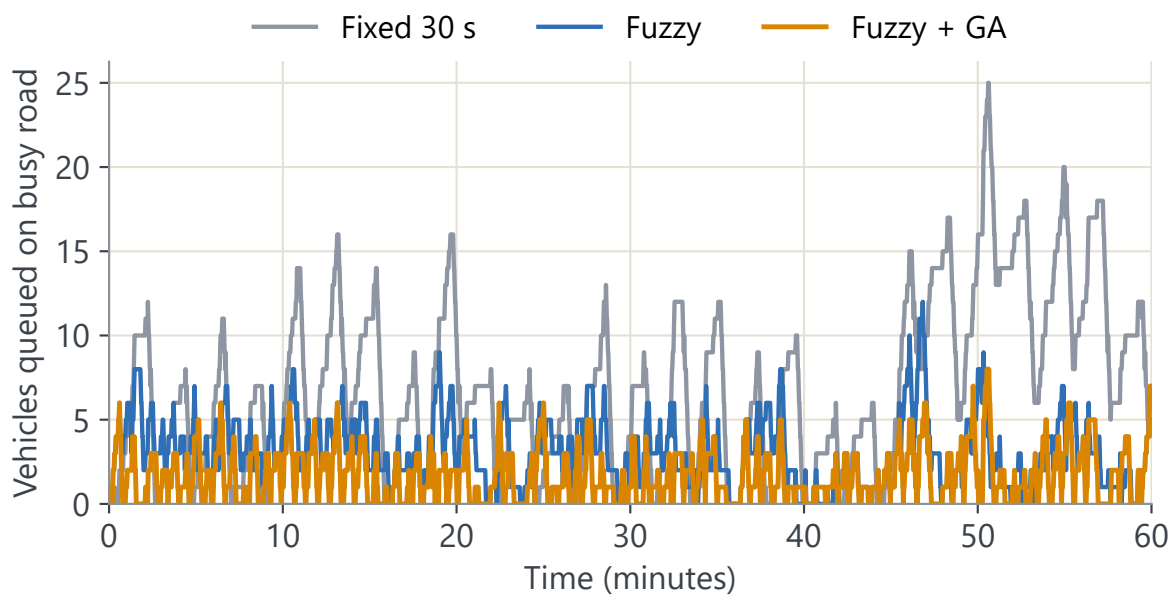
Waiting roads: more sensitive

"Medium" now peaks at 3 waiting vehicles instead of 10. The controller reacts to waiting traffic much sooner.

Extension: shorter steps

The "Medium" extension peaks at 6.5 s instead of 10 s. Shorter, more frequent decisions waste less green.

Watching the busy road for one hour



Unbalanced scenario, one unseen run, road with 360 vehicles per hour

Fixed timer: long queue spikes

The busy road gets the same 30 s as the quiet roads, so its queue climbs to 25 vehicles at the worst point in this run.

Fuzzy: queue stays controlled

Green time follows demand. The longest queue is 12 vehicles hand-set and 8 after GA tuning.

The tuned controller is also more **consistent**: across the 10 runs the standard deviation of its waiting time is 0.4 s, against 5.1 s for the fixed timer.

Conclusion

1. **All five objectives were met.** Simulator, fuzzy controller, GA tuning, three-way comparison and three traffic scenarios are implemented and measured.

2. **Fuzzy control beats fixed timing** in every scenario, cutting average waiting by 25% to 61% even with hand-set membership functions.

3. **Optimization matters.** GA tuning removed a further 11% to 35%, giving a total reduction of 50% to 71% against the 30 s fixed timer.

4. **The biggest gain is where traffic is uneven,** which is exactly the case a fixed timer cannot handle.

5. **The result generalises.** The tuned controller performs the same on unseen traffic as on the traffic it was trained on.

Limitations and future scope

Honest limitations

- Simulated traffic, not real sensor data
- One isolated junction with a fixed phase order
- No turning lanes, pedestrians or emergency vehicles
- Fixed baselines use equal green for all roads
- The rule base itself was not optimized, only the membership functions

Future scope

- Validate on SUMO with real junction counts
- Let the controller skip empty roads and choose the next phase
- Compare GA with PSO on the same tuning task
- Add emergency vehicle priority as an extra fuzzy input
- Extend to a corridor of connected junctions

Take-away: a small set of human-readable rules, tuned automatically, already halves waiting time in this model. The same idea can scale to real junctions once sensor data is available.

References

[1] F. V. Webster, "Traffic Signal Settings," Road Research Technical Paper No. 39, HMSO, London, 1958.

[2] L. A. Zadeh, "Fuzzy sets," *Information and Control*, vol. 8, no. 3, pp. 338-353, 1965.

[3] C. P. Pappis and E. H. Mamdani, "A fuzzy logic controller for a traffic junction," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 7, no. 10, pp. 707-717, 1977.

[4] P. B. Hunt, D. I. Robertson, R. D. Bretherton and R. I. Winton, "SCOOT: A traffic responsive method of coordinating signals," TRRL Laboratory Report 1014, 1981.

[5] C. L. Karr, "Genetic algorithms for fuzzy controllers," *AI Expert*, vol. 6, no. 2, pp. 26-33, 1991.

[6] M. B. Trabia, M. S. Kaseko and M. Ande, "A two-stage fuzzy logic controller for traffic signals," *Transportation Research Part C*, vol. 7, no. 6, pp. 353-367, 1999.

[7] J. Niittymaki and M. Pursula, "Signal control using fuzzy logic," *Fuzzy Sets and Systems*, vol. 116, no. 1, pp. 11-22, 2000.

[8] J. H. Holland, *Adaptation in Natural and Artificial Systems*, University of Michigan Press, 1975.

Thank you



I am happy to take your questions.

Mohammed Amaan Khan

25WU0101075 | B.Tech CSE, 2nd Year | Rhinos | Guided by Dr. Bhargav Prajwal Pathri